

# From MLPs to Trainable Models

VLC =  $\int \int \int$  Vision  
Learning *vLaC*  
Control

## From MLPs to Trainable Models

Activations, objectives, and initialisation

Jonathon Hare and Antonia Marcu

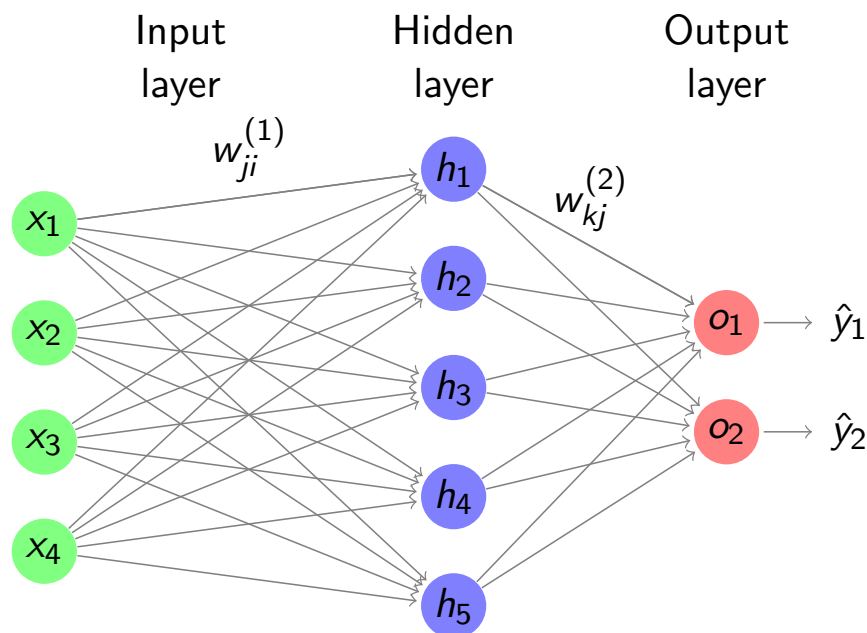
Vision, Learning and Control  
University of Southampton

- Multilayer perceptrons as compositions of affine maps and nonlinearities
- Activations, logits, and loss functions
- The complete training pipeline
- Initialisation and the scale of signals and gradients
- Input scaling, weight initialisation, and internal normalisation

## How Supervised Learning Typically Works

- Start by choosing a model-class:  $\hat{y} = f(\mathbf{x}; \mathbf{W})$  where the model-class  $f$  is a way of using some numerical parameters,  $\mathbf{W}$ , to map each input vector  $\mathbf{x}$  to a predicted output  $\hat{y}$ .
- *Learning* means adjusting the parameters to reduce the discrepancy between the true target output  $y$  on each training case and the output  $\hat{y}$ , predicted by the model.

## Let's look at a Multilayer Perceptron (without biases)...



Without loss of generality, we can write the above as:

$$\hat{\mathbf{y}} = g(f(\mathbf{x}; \mathbf{W}^{(1)}); \mathbf{W}^{(2)}) = g(\mathbf{W}^{(2)} f(\mathbf{W}^{(1)} \mathbf{x}))$$

where  $f$  and  $g$  are activation functions.

## Common Activation Functions

- Identity
- Sigmoid (aka Logistic)
- Hyperbolic Tangent (tanh)
- Rectified Linear Unit (ReLU) (aka Threshold Linear)

$$\hat{\mathbf{y}} = g(\mathbf{W}^{(2)} f(\mathbf{W}^{(1)} \mathbf{x}))$$

- What form should the final layer function  $g$  take?
- It depends on the task (and on the chosen loss function)...
  - regression  $\rightarrow$  typically linear
  - binary classification  $\rightarrow$  typically Sigmoid
  - multilabel classification  $\rightarrow$  typically Sigmoid
  - multiclass classification  $\rightarrow$  typically Softmax

## Softmax

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad \forall i = 1, 2, \dots, K$$

- The outputs are positive and sum to 1.
- The denominator contains every logit, so every output depends on every input.
- Let  $\mathbf{s} = \text{softmax}(\mathbf{z})$ . The entries of its Jacobian are:

$$\frac{\partial s_i}{\partial z_j} = s_i(\delta_{ij} - s_j).$$

Here  $\delta_{ij}$  is the Kronecker delta: it is 1 if  $i = j$ , and 0 otherwise.

# Softmax as a relaxation of one-hot argmax

- A multiclass decision selects the largest logit:

$$c = \arg \max_k z_k.$$

- The equivalent one-hot vector is discontinuous: a small change in the logits can switch the selected class.
- Softmax replaces the hard one-hot vector with a continuous, differentiable distribution:

$$\begin{aligned} \mathbf{z} &= \begin{bmatrix} 1.1 & 4.0 & -0.1 & 2.3 \end{bmatrix} \\ \text{onehot}(\arg \max_k z_k) &= \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix} \\ \text{softmax}(\mathbf{z}) &= \begin{bmatrix} 0.044 & 0.797 & 0.013 & 0.146 \end{bmatrix} \end{aligned}$$

- The largest component stays associated with the predicted class, but the output retains information about the alternatives.

## Temperature controls the relaxation

$$\text{softmax}(\mathbf{z}/T)_i = \frac{e^{z_i/T}}{\sum_{j=1}^K e^{z_j/T}}, \quad T > 0.$$

- Lower temperatures sharpen the distribution:

$$\begin{aligned} T = 1.0 : & \begin{bmatrix} 0.044 & 0.797 & 0.013 & 0.146 \end{bmatrix} \\ T = 0.6 : & \begin{bmatrix} 0.008 & 0.937 & 0.001 & 0.055 \end{bmatrix} \\ T = 0.2 : & \begin{bmatrix} 5.0 \times 10^{-7} & 0.9998 & 1.3 \times 10^{-9} & 0.0002 \end{bmatrix} \end{aligned}$$

- If the largest logit is unique, then as  $T \rightarrow 0^+$ :

$$\text{softmax}(\mathbf{z}/T) \longrightarrow \text{onehot}(\arg \max_k z_k).$$

- Higher temperatures flatten the distribution. As  $T \rightarrow \infty$ , it approaches the uniform distribution.

# Ok, so let's talk loss functions

- The choice of loss function depends on:
  - 1 the task (e.g. classification/regression/something else)
  - 2 the activation function of the last layer
    - For numerical reasons  
the activation is often computed directly within the loss rather than being part of the model
- Some classification losses require *raw outputs* (e.g. a linear layer) of the network as their input
  - These are often called *unnormalised log probabilities* or *logits*
  - An example would be hinge-loss used to create a Support Vector Machine for binary classification
- There are many different loss functions we might encounter (MSE, Cross-Entropy, KL-Divergence, huber, L1 (MAE), CTC, Triplet, ...) for different tasks.

## The Loss Function (measure of discrepancy)

Recall from Foundations of Machine Learning:

- Mean Squared Error (MSE) loss for a single data point is given by
$$\ell_{MSE}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_i (\hat{y}_i - y_i)^2 = (\hat{\mathbf{y}} - \mathbf{y})^\top (\hat{\mathbf{y}} - \mathbf{y})$$
- We often multiply this by a constant factor of  $\frac{1}{2}$  — can anyone guess/remember why?
- $\ell_{MSE}(\hat{\mathbf{y}}, \mathbf{y})$  is the predominant choice for regression problems with linear activation in the last layer
- For a classification problem with Softmax or Sigmoidal activations MSE can cause slow learning
  - Gradients of  $\ell_{MSE}$  are proportional to the difference in target and predicted value, multiplied by the gradient of the activation function
  - **The Cross-Entropy loss function is generally a better choice in this case**

For the binary classification case:

$$\ell_{BCE}(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

- The cross-entropy loss function is non-negative,  $\ell_{BCE} > 0$
- $\ell_{BCE} \approx 0$  when the prediction and targets are equal (i.e.  $y = 0$  and  $\hat{y} = 0$  or when  $y = 1$  and  $\hat{y} = 1$ )
- With Sigmoidal final layer,  $\frac{\partial \ell_{BCE}}{\partial \mathbf{w}_i^{(2)}}$  is proportional to just the error in the output ( $\hat{y} - y$ ) and therefore, the larger the error, the faster the network will learn!
- Note that the BCE is the negative log likelihood of the Bernoulli Distribution

## Binary Cross-Entropy — Intuition

- The cross-entropy can be thought of as a **measure of surprise**.
- Given some input  $x_i$ , we can think of  $\hat{y}_i$  as the estimated probability that  $x_i$  belongs to class 1, and  $1 - \hat{y}_i$  is the estimated probability that it belongs to class 0.
- Note the extreme case of infinite cross-entropy, if your model believes that a class has 0 probability of occurrence, and yet the class appears in the data, the 'surprise' of your model will be infinitely great.

# Binary Cross-Entropy for multiple labels

In the case of multi-label classification with a network with multiple sigmoidal outputs you just sum the BCE over the outputs:

$$\ell_{BCE} = - \sum_{k=1}^K [y_k \log(\hat{y}_k) + (1 - y_k) \log(1 - \hat{y}_k)]$$

where  $K$  is the number of classes of the classification problem,  $\hat{y} \in \mathbb{R}^K$ .

## Numerical Stability: The Log-Sum-Exp trick

$$\ell_{BCE}(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

- Consider what might happen early in training when the model might confidently predict a positive example as negative
  - $\hat{y} = \sigma(z) \approx 0 \implies z \ll 0$
  - if  $\hat{y}$  is small enough, it will become 0 due to limited precision of floating-point representations
  - but then  $\log(\hat{y}) = -\infty$ , and everything will break!
- To tackle this problem implementations usually combine the sigmoid computation and BCE into a single loss function that you would apply to a network with linear outputs (e.g. `BCEWithLogitsLoss`).
- Internally, a trick called ‘log-sum-exp’ is used to *shift* the centre of an exponential sum so that only numerical underflow can potentially happen, rather than overflow
- - Ultimately this means you’ll always get a numerically reasonable result (and will avoid NaNs and Infs originating from this point).

# Multiclass classification with Softmax Outputs

- Softmax can be thought of making the  $K$  outputs of the network mimic a probability distribution.
- The target label  $y$  could also be represented as a distribution with a single 1 and zeros everywhere else.
  - e.g. they are “one-hot encoded”.
- In such a case, the obvious loss function is the *negative log likelihood* of the Categorical distribution (aka Multinoulli, Generalised Bernoulli, Multinomial with one sample)
  - Note that in practice as  $y_k$  is zero for all but one class you don't actually do this summation, and if  $y$  is an integer class index you can write  $\ell_{NLL} = -\log \hat{y}_y$ .

Log-Sum-Exp can be used for better numerical stability. PyTorch combines LogSoftmax with NLL in one loss and calls this “Categorical Cross-Entropy” (so you would use this with a **linear output layer**)

## The complete training pipeline

- 1 Initialise the model parameters  $\theta$  and the optimiser state.
- 2 Draw a minibatch  $(\mathbf{X}, \mathbf{y})$  and run the forward pass:

$$\hat{\mathbf{y}} = f(\mathbf{X}; \theta), \quad \mathcal{L} = \frac{1}{B} \sum_{i=1}^B \ell(\hat{\mathbf{y}}_i, \mathbf{y}_i).$$

- 3 Use reverse-mode automatic differentiation to compute  $\nabla_{\theta} \mathcal{L}$ .
- 4 Update the parameters and repeat with another minibatch.

### One PyTorch training step

```
optimizer.zero_grad() → loss = criterion(model(x), y) →  
loss.backward() → optimizer.step()
```

# Why zero initialisation fails

- Suppose every hidden unit in a layer starts with the same incoming weights and bias.
- The units compute the same activation and receive the same gradient:

$$\mathbf{w}_j = \mathbf{w}_k \implies h_j = h_k \quad \text{and} \quad \nabla_{\mathbf{w}_j} \mathcal{L} = \nabla_{\mathbf{w}_k} \mathcal{L}.$$

- An update preserves this equality. The units remain copies of one another, so the layer cannot learn distinct features.
- Random weight initialisation breaks this symmetry.
  - Biases may still be initialised to zero because the random weights have already broken the symmetry.
- With ReLU, zero pre-activations are especially problematic because PyTorch uses a derivative of zero at the origin.

# The scale of the initial weights matters

- Each layer transforms the scale of its inputs. Let

$$q_\ell = \mathbb{E}[(h^{(\ell)})^2], \quad q_\ell \approx c \, q_{\ell-1}.$$

- Across  $L$  similar layers:

$$q_L \approx c^L q_0.$$

- If  $c = 0.5$ , then after 20 layers the signal is scaled by about  $10^{-6}$ .
- If  $c = 2$ , the same network amplifies it by about  $10^6$ .
- Backpropagation has the same repeated-multiplication problem. We want activations and gradients to stay at a useful scale.

# Variance propagation through a linear layer

For one output of a linear layer,

$$z_j = \sum_{i=1}^{n_{\text{in}}} w_{ji} x_i.$$

- Assume the inputs and weights are independent, zero-centred, and identically distributed.
- If  $\text{Var}[w_{ji}] = \sigma_w^2$  and  $\text{Var}[x_i] = q$ , then

$$\text{Var}[z_j] = n_{\text{in}} \sigma_w^2 q.$$

- Preserving the forward variance suggests  $\sigma_w^2 \approx 1/n_{\text{in}}$ .
- Applying the same argument to the backward pass suggests  $\sigma_w^2 \approx 1/n_{\text{out}}$ .

## Xavier and Glorot initialisation

- When  $n_{\text{in}} \neq n_{\text{out}}$ , one weight variance cannot exactly preserve scale in both directions.
- Xavier initialisation uses the compromise

$$\text{Var}[w] = \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

- Common forms are

$$w \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

or

$$w \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right].$$

- This reasoning fits linear activations and tanh near its approximately linear region.

# Why Xavier under-scales ReLU networks

Let  $q_\ell = \mathbb{E}[(h^{(\ell)})^2]$ . If the pre-activations are symmetric around zero, then

$$\mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2}\mathbb{E}[z^2].$$

- With equal fan-in and fan-out, Xavier gives  $\text{Var}[w] \approx 1/n_{\text{in}}$ .
- A ReLU layer therefore gives approximately

$$q_\ell \approx \frac{1}{2} n_{\text{in}} \text{Var}[w] q_{\ell-1} \approx \frac{1}{2} q_{\ell-1}.$$

- Repeated ReLU layers can shrink the activation scale by roughly  $2^{-L}$ . The backward pass loses a similar factor because only the active ReLUs transmit gradients.

## He and Kaiming initialisation

- He initialisation compensates for the factor of  $\frac{1}{2}$  introduced by ReLU:

$$\text{Var}[w] = \frac{2}{n_{\text{in}}}.$$

- Now the activation second moment is approximately preserved:

$$q_\ell \approx \frac{1}{2} n_{\text{in}} \frac{2}{n_{\text{in}}} q_{\ell-1} = q_{\ell-1}.$$

- Common choices are

$$w \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right) \quad \text{or} \quad w \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right].$$

- The correct gain depends on the activation. Leaky ReLU, for example, also passes part of the negative signal.

# Bias initialisation and PyTorch defaults

- Hidden-layer biases can usually start at zero. Random weights already break the symmetry between units.
- An output bias can encode a useful prior. For binary classification with positive rate  $p$ :

$$b = \log \frac{p}{1-p}.$$

- `nn.Linear` currently draws its weights and biases uniformly from

$$\left[ -\frac{1}{\sqrt{n_{\text{in}}}}, \frac{1}{\sqrt{n_{\text{in}}}} \right].$$

This historical default is not the standard He initialisation for a ReLU layer.

- Initialise explicitly when the activation assumption matters:

```
nn.init.kaiming_normal_(layer.weight,  
                        nonlinearity="relu")  
nn.init.zeros_(layer.bias)
```

PyTorch `nn.Linear` source

Jonathon Hare and Antonia Marcu

AICE3002 / AICE6001

25 / 26

## Input scaling, initialisation, and internal normalisation

| Mechanism                     | What it controls                                                | When it acts                                                                |
|-------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Input scaling</b>          | The scale and conditioning of the data entering the first layer | Before training. Estimate scaling statistics from the training data only.   |
| <b>Weight initialisation</b>  | The starting scale and symmetry of the parameters               | Once, when the model is constructed. Match it to the activation.            |
| <b>Internal normalisation</b> | The scale of learned activations inside the network             | During each forward pass. BatchNorm and LayerNorm use different statistics. |

- These mechanisms complement one another. Normalisation does not make careful data scaling or sensible initialisation irrelevant.
- We will return to internal normalisation in the practical training